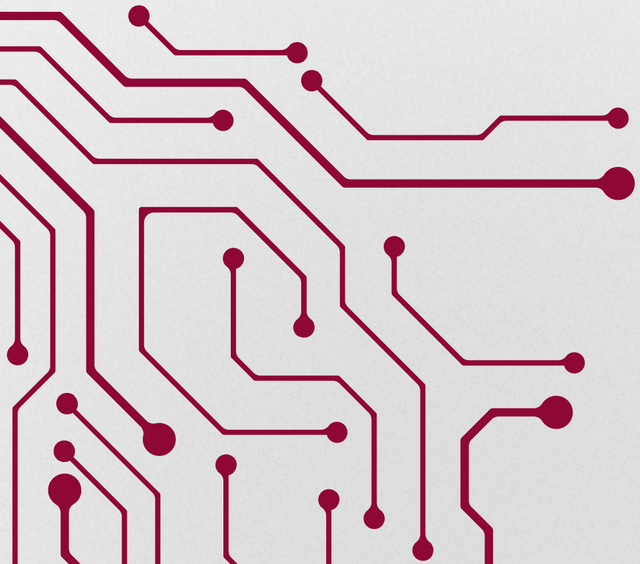
CONVOLUTIONAL NEURAL NETWORK



For ISC4933 Computational Probabilistic Modeling

presented by Bruna Lelis do Prado | April 2026

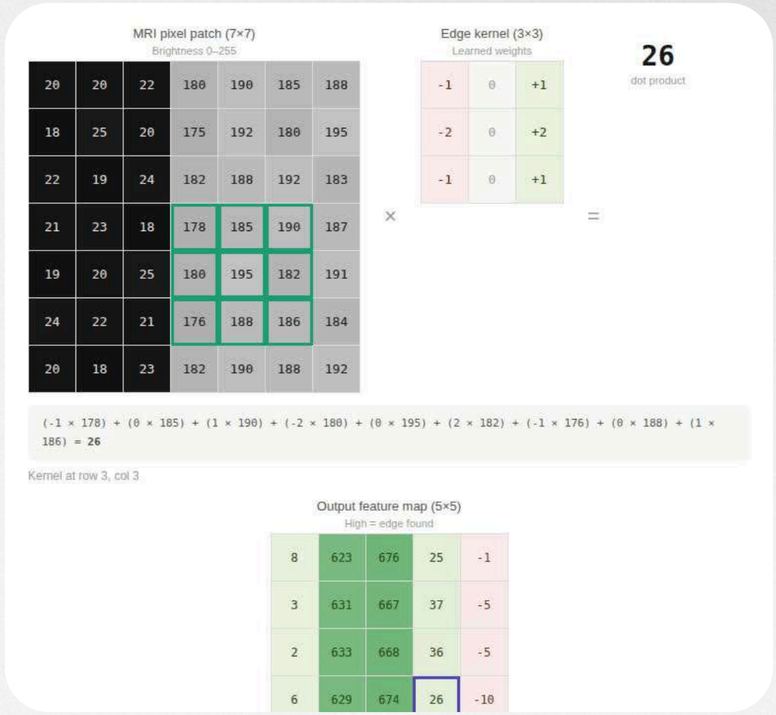
WHAT IS CONVOLUTION?

Convolution is a mathematical operation that slides a small matrix (a Kernel) over a larger matrix (the Image). It acts like a "magnifying glass," looking for one specific thing at a time.

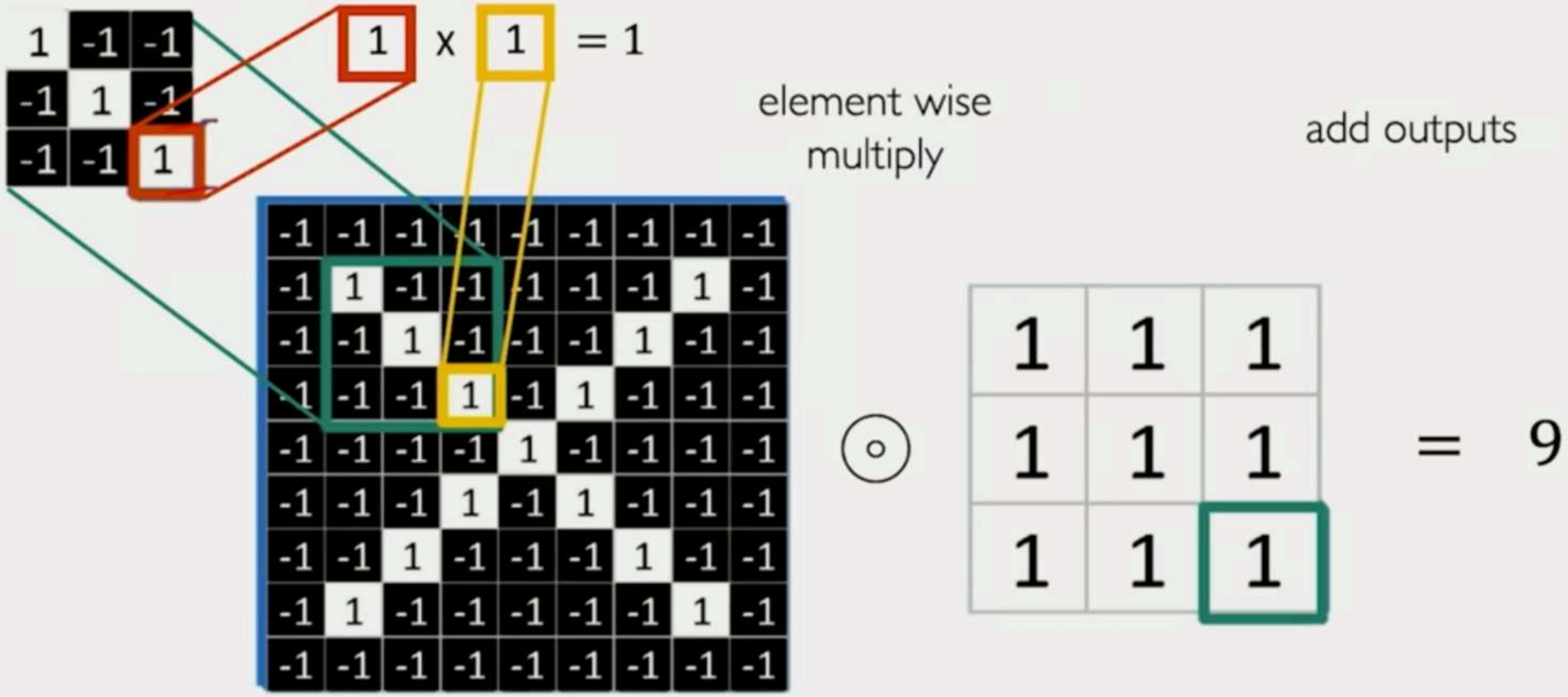
The kernel values are multiplied by the pixel values they are currently "hovering" over. You add all those products together to get a single number.

That single number becomes one pixel in a new, smaller image called a Feature Map.

- High number? The pattern was found!
- Low number? No match here.



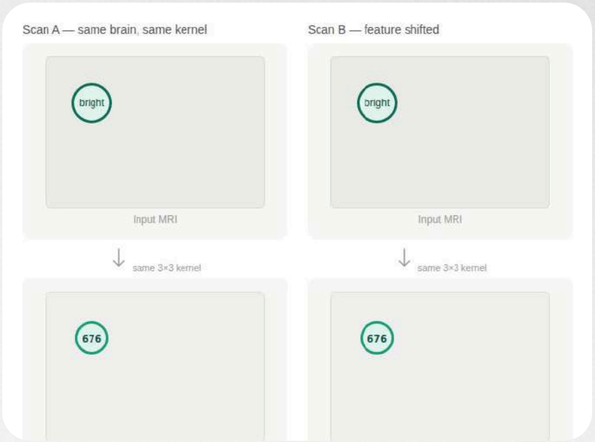
The Convolution Operation

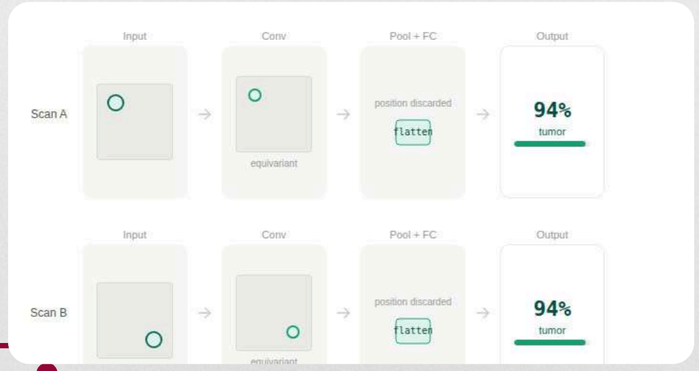


BASIC PROPERTIES OF CONVOLUTION

Local connectivity: In a standard network, every input connects to every output. In a CNN, a neuron only responds about a small neighborhood of pixels. This dramatically reduces the number of weights (parameters) we need to optimize, preventing overfitting.

Shift equivariance: The same kernel scans every position. If a feature moves, its detection moves with it; nothing else changes. (Same detector, every location)

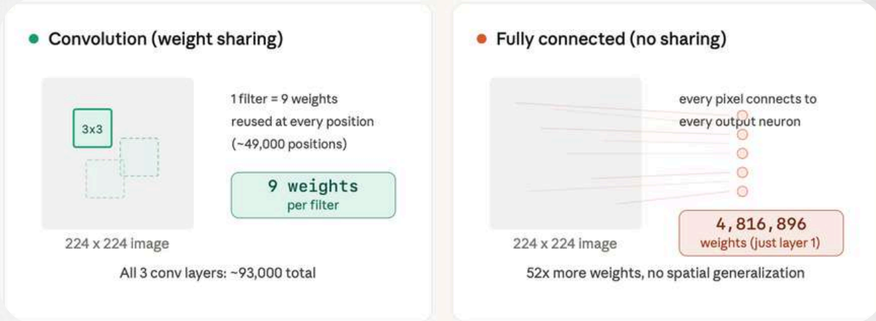




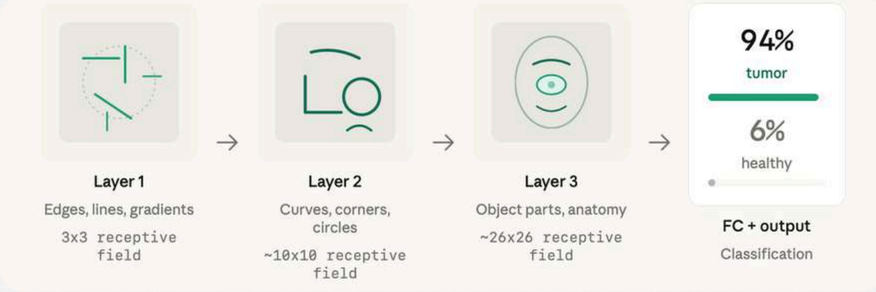
Translation Invariance: Even if an object shifts position in the image, the network still recognizes it. Pooling layer helps achieve this by summarizing local regions, so the output stays the same regardless of where the feature appears.

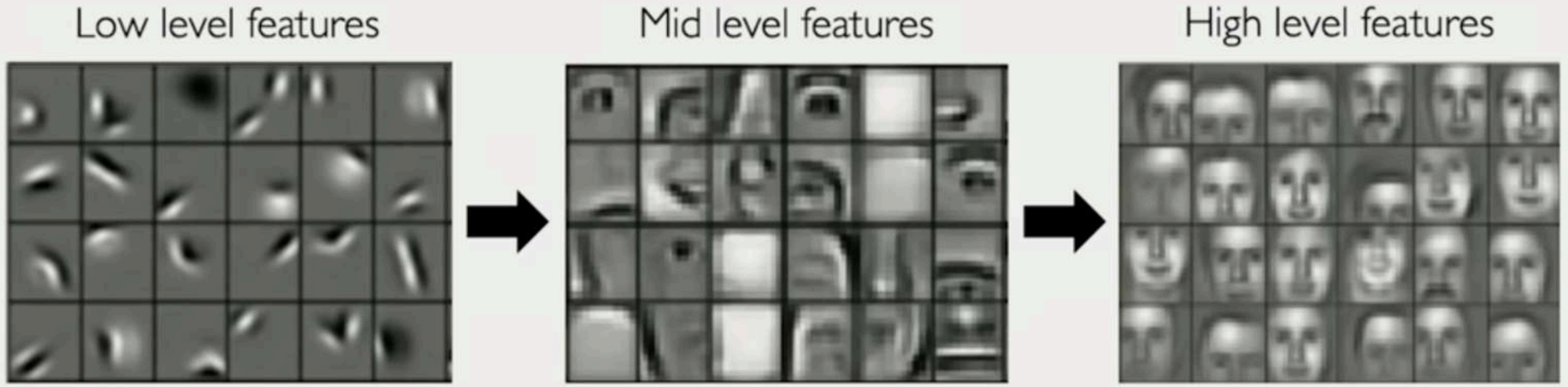
BASIC PROPERTIES OF CONVOLUTION 2.0

Parameter efficiency: A 3×3 kernel has only 9 learnable weights, and those same 9 weights are reused at every position across the image. A Fully Connected (FC) layer, where every input pixel connects to every output neuron, would need millions of weights to do the same job on a 224×224 image. Weight sharing is what makes CNNs so much more efficient.



Hierarchical features: Layer 1 sees 3x3-pixel patches and learns edges. Layer 2 sees combinations of edges and learns curves and corners. Layer 3 sees combinations of shapes and learns anatomical structures. Each layer's receptive field grows, so each layer can detect larger, more meaningful patterns. (Edges → Shapes → Structures)





CONVOLUTION AS A SLIDING WINDOW

A small grid of numbers, the kernel/filter, slides across the image one position at a time. At each stop, it multiplies its weights against the pixels underneath and sums the result into a single output value. That output tells you how strongly the pattern matched at that location.

The kernel never changes as it moves. The same 9 weights that checked the top-left corner check every other corner, edge, and center patch. One pass across a 224×224 image means roughly 49,000 stops each producing one value in the output feature map.

Discrete 2D cross-correlation

(called "convolution" in deep learning)

$$(f \star k)(i,\, j) \;=\; \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(i+m,\; j+n) \;\cdot\; k(m,\, n)$$

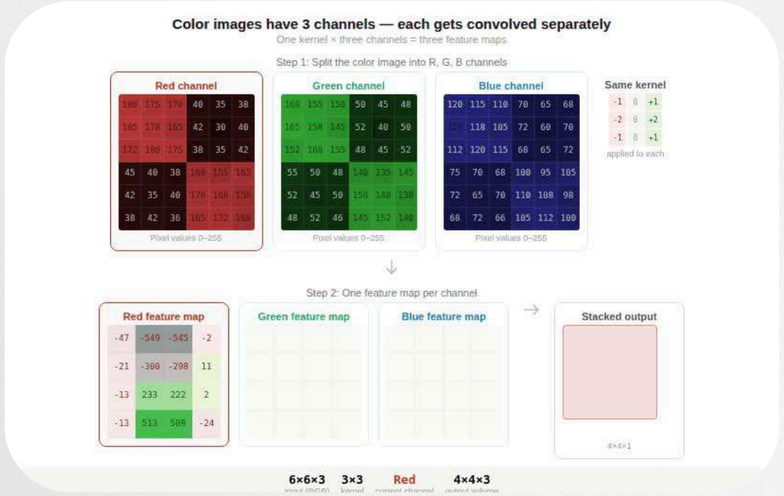
f	=	input image (pixel values)
k	=	kernel / filter (learned weights)
(i, j)	=	output position
(m, n)	=	position within the kernel
$M \times N$	=	kernel size (e.g. 3×3)

APPLYING CONVOLUTIONS TO IMAGES

An image is two buckets of numbers. Bucket one is the image itself, three grids of pixel values stacked together, one for red, one for green, one for blue. Each value is an integer from 0 (no color) to 255 (full color). A grayscale MRI uses just one grid.

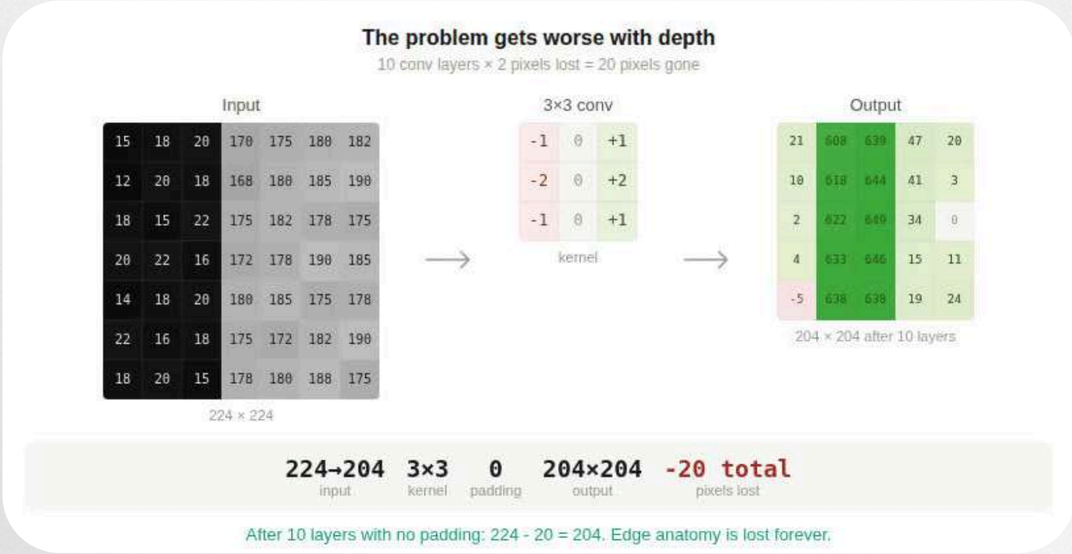
Bucket two is the kernel, a small grid of decimal numbers that acts like a recipe. The pattern and size of these numbers determine what the kernel detects: edges, textures, spots, or gradients.

Convolution mixes the two buckets. Place the kernel on top of a patch of pixels, multiply each pair, sum the products, write the result into the output. Slide one pixel over, repeat. The output is a feature map.



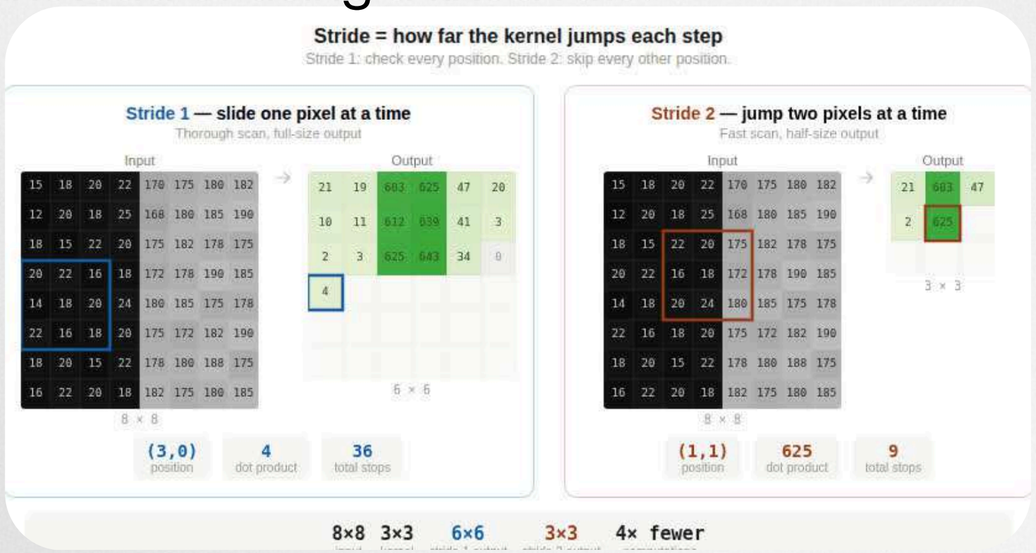
PADDING & STRIDE

The shrinking problem:
A 3x3 kernel on a 224x224 image produces 222x222 output; we lose 2 pixels per side because the kernel can't center on edge pixels. Stack 10 cone layers with no fix, and the image shrinks from 224 to 204. Padding solves this: wrap the image in a border of zeros before convolving. One pixel of zero-padding on each side turn 224 into 226, and after the 3x3 kernel, we are back to 224x224. The output stays the same size as the input.



PADDING & STRIDE 2.0

The skip option:
By default, the kernel moves one pixel at a time, stride 1. Set stride to 2, and the kernel jumps two pixels per step, cutting the output dimensions in half. A stride-2 conv on a 224x224 image produces a 112x112 output. This is an alternative to pooling for downsampling; some modern architectures replace max pooling entierely with stride-2 convolutions because the network can learn how to downsample instead of using a fixed rule.



Stride controls how fast the kernel moves. Padding controls whether edges get included. Together, they decide the size of the output, and tuning them is how architects control information flow through the network.

FEATURE MAPS

One kernel scans the image → one feature map.

High values = pattern found
Zero = nothing there

32 kernels scan the same image → 32 feature maps stacked into a volume.

Layer 2 reads all 32 maps at once, combining simple patterns into complex ones.

CONVOLUTION IN NEURAL NETWORK

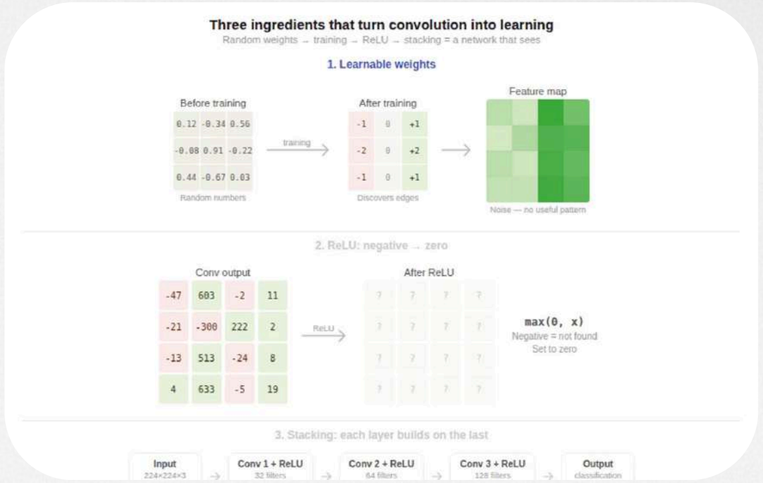
Random numbers become pattern detectors through thousands of rounds of trial and correction.

How does it learn?

- The kernel starts with random weights, it has no idea what it's looking for
- It scans the image and makes a prediction, forward pass
- A loss function scores how wrong it was
- Backpropagation figures out which weights made it worse
- Each weight is adjusted to do better next time, weight update
- Repeat thousands of times → the kernel starts detecting real patterns

Why stack layers?

- After each convolution, ReLU wipes all negative values to zero
- Without it, all layers would do the same job, just one layer pretending to be many
- ReLU gives each layer its own thing to learn



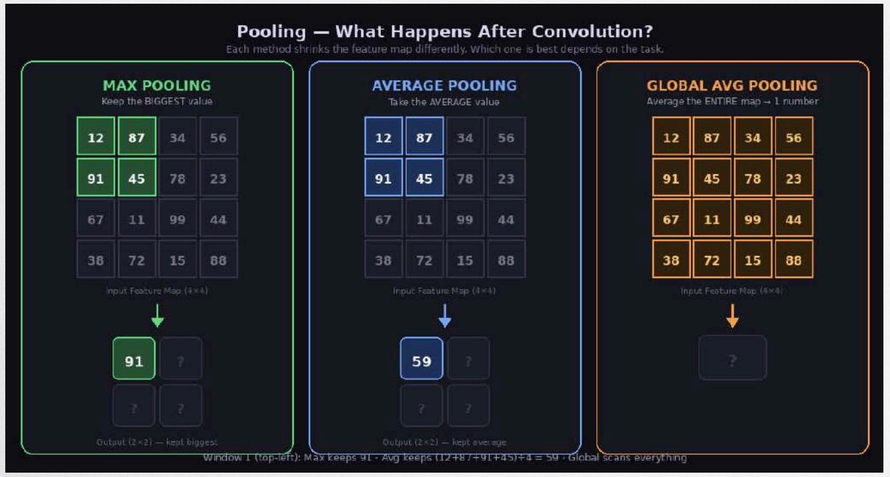
Forward pass → loss → backpropagation → weight update → repeat. This is how a network learns to see.

ACTIVATION AND POOLING LAYERS

After every convolution, ReLU decides what's worth keeping. Positive value = a pattern was detected → keep it. Negative value = nothing meaningful here → set it to zero.

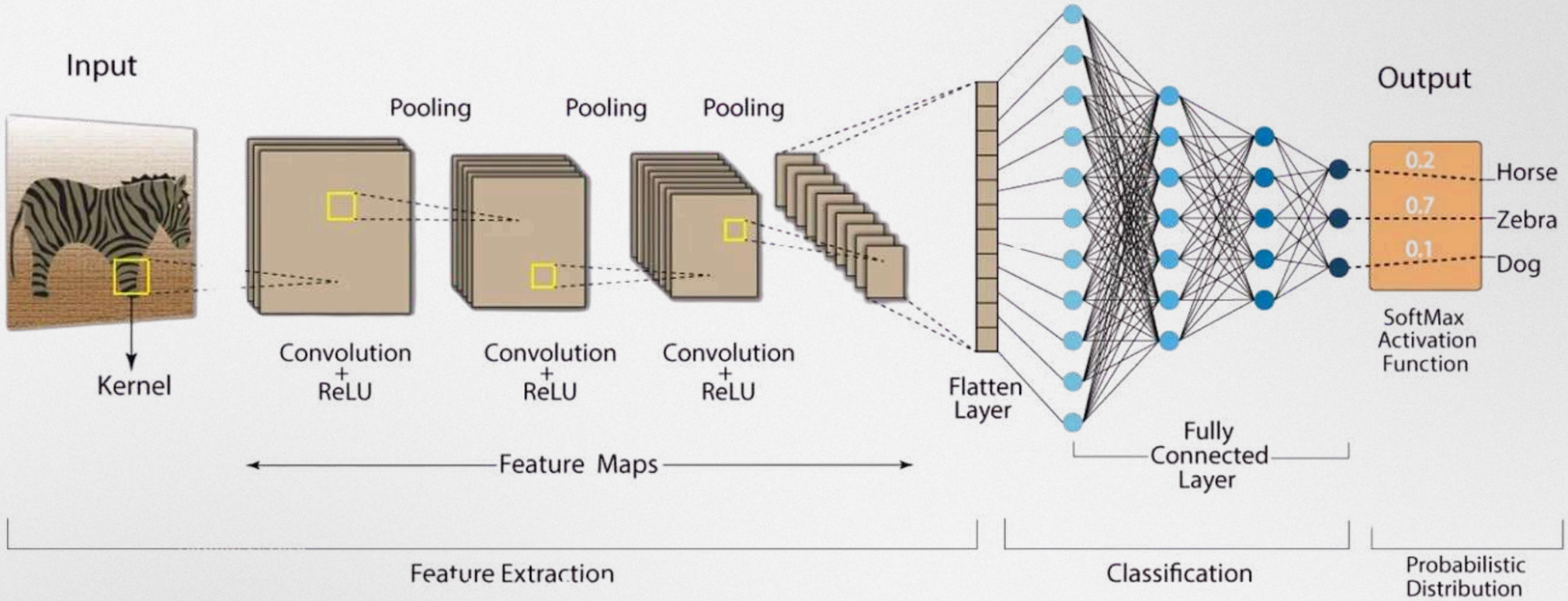
$$\mathrm{ReLU}(x) = \max(0,\ x)$$

After ReLU, Max Pooling zooms out. It slides a 2×2 window across the feature map and keeps only the strongest signal in each patch. The map shrinks from 222×222 → 111×111. We're not losing the important stuff, we're throwing away the redundant detail.



Conv finds patterns. ReLU keeps the strong ones. Pool zooms out. Repeat until the network sees the full picture.

Convolution Neural Network (CNN)



Input - brain MRI
A 224x224 grayscale MRI scan with 1 channel (no RGB, just brightness). Each pixel is a value from 0 (black) to 255 (white)

Conv + ReLU
Kernels scan the MRI looking for patterns. ReLU removes negatives so each layer learns something new. Repeated 3x Edges → brain regions → anatomical structures.

Max pooling
Slides a 2x2 window and keeps only the highest value in each region. Shrinks the image in half, keeps the important stuff, drops the rest.

Flatten
The 3D volume (52x52x128) gets unrolled into a single list of 346,112 numbers. Nothing is lost, we just reshaped because all our FC layer can handle is 1D input.

Output (2 classes)
2 possible answers: Tumor or No Tumor. The network outputs a score for each and picks the highest confidence one.

BENEFITS OF CNNS

- Automatic feature learning. CNNs learn what to look for directly from raw pixel data. Nobody hand-designs the edge detectors or texture filters; the network discovers them through training. A standard neural network needs hand-engineered features as input.

- Parameter sharing. In a standard neural network, every neuron has its own separate set of weights, millions of unique parameters that are each used once. CNNs reuse the same filter across the entire image. This makes the network dramatically more efficient and less prone to overfitting, because there are far fewer parameters to memorize noise with.

- Local connectivity. Each neuron only processes a small region of the input at a time (a 3×3 patch, not the full 224×224 image). This forces the network to focus on spatially relevant features and capture local patterns first. Building up gradually rather than trying to understand the whole image at once.

APPLICATIONS OF CNNS

For our presentation, as you saw, we used a CNN to process an MRI brain scan from raw pixels to a tumor diagnosis. That same architecture powers applications across every field.

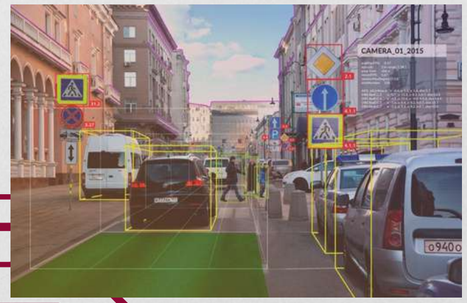
Medical imaging: MRI reconstruction from undersampled scans, tumor detection in X-rays, retinal disease screening, and pathology slide analysis.

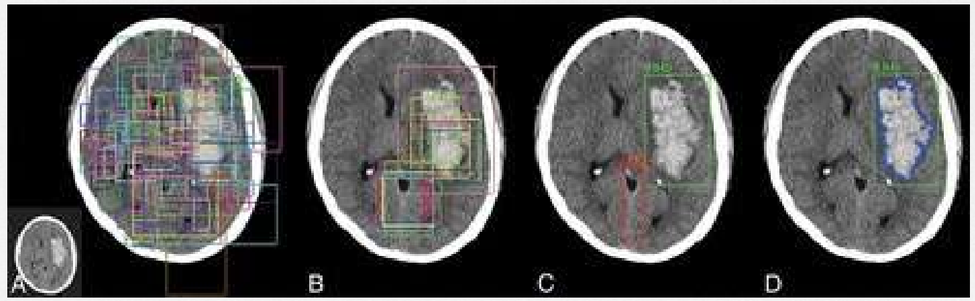
Autonomous vehicles: Pedestrian detection, lane tracking, traffic sign recognition.

Photo and video: Phone photo search, face tagging, and content moderation on social media.

Security: Surveillance anomaly detection, facial recognition, license plate reading.

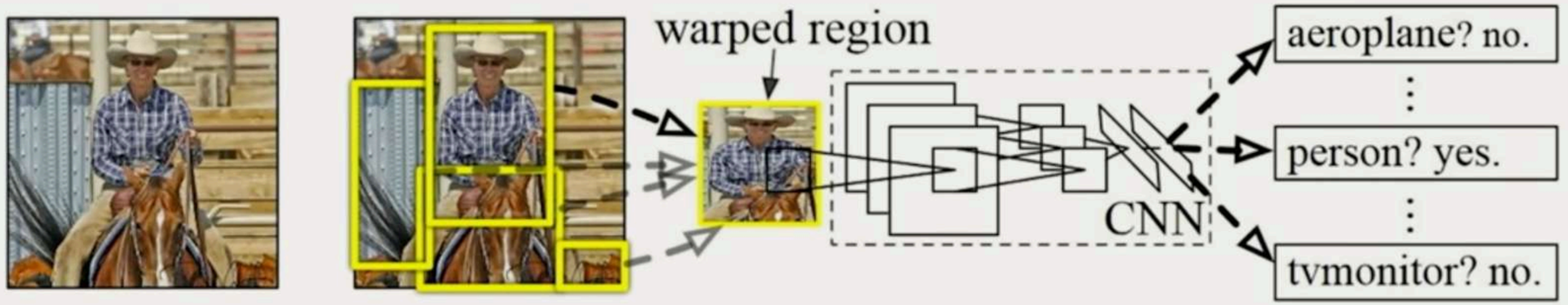
Manufacturing: Defect detection on assembly lines, quality control inspection.





Object Detection with R-CNNs

R-CNN algorithm: Find regions that we think have objects. Use CNN to classify.



1. Input image

2. Extract region proposals (~2k)

3. Compute CNN features

4. Classify regions

SOURCES

Dettmers, T. (2015). Understanding Convolution in Deep Learning. timdettmers.com/2015/03/26/convolution-deep-learning/

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press. Chapter 9: Convolutional Networks.

NVIDIA. (n.d.). Convolution. NVIDIA Developer. https://developer.nvidia.com/discover/convolution

IBM Technology. (2023). What are Convolutional Neural Networks (CNNs)? [Video]. YouTube. https://www.youtube.com/watch?v=QzY57FaENXg

Amidi, A., & Amidi, S. (n.d.). Convolutional Neural Networks Cheatsheet. Stanford University, CS 230 — Deep Learning. https://stanford.edu/~shervine/teaching/cs-230/cheatsheet-convolutional-neural-networks/

Kourounis, G., Elmahmudi, A. A., Thomson, B., Hunter, J., Ugail, H., & Wilson, C. (2023). Computer image analysis with artificial intelligence: A practical introduction to convolutional neural networks for medical professionals. Postgraduate Medical Journal, 99(1178), 1287–1294. https://doi.org/10.1093/postmj/qgad095

Amini, A. (2025). MIT 6.S191: Convolutional Neural Networks [Video]. YouTube. https://www.youtube.com/watch?v=oGpzWAlP5p0

Vishnu, A. M. (n.d.). To explain the loss functions in a convolutional neural network (CNN). Medium. https://medium.com/@vishnuam/to-explain-the-loss-functions-in-a-convolutional-neural-network-cnn-a86573ac4bc4

IBM. (n.d.). Loss function. IBM Think. https://www.ibm.com/think/topics/loss-function

Gur Arie, L. (n.d.). Neural networks: Backpropagation. Medium. https://medium.com/data-science/neural-networks-backpropagation-by-dr-lihi-gur-arie-27be67d8fdce